Cloud Computing (CS3081) Mini Project Report

End-to-End Sequence-Level Knowledge Distillation, Containerized FastAPI Inference, and Kubernetes Deployment

Student: Yash Chahar | **GitHub:** yograjchahar05/cc-assignment | **Release Tag:** v1.0.0 (Asset verified) | **Final Commit:** final

Executive Summary & System Architecture

This project implements a complete cloud-native machine learning deployment lifecycle for a lightweight voice-command parser. A large teacher model (Phi-3.5, 3.8B parameters) was queried via parallel Kubernetes worker pods to label raw natural language utterances. The generated dataset was cleaned via strict mathematical validation and used to train a lightweight student language model (EleutherAI/pythia-70m) through Sequence-Level Knowledge Distillation exclusively on CPU. The student was packaged into an optimized, CPU-only FastAPI container (<2 GB) that downloads model weights dynamically from S3-compatible SeaweedFS object storage. The microservice was deployed onto a local k3d Kubernetes cluster with HPA autoscaling, health/readiness probes, and Traefik ingress, and subjected to stateless subnetting, NACL security verification, automated CI/CD quality gating, and Locust load testing.

1. Task 1: Object Storage and Parallel Teacher Labelling

1.1 Kubernetes Cluster & Object Storage: A local k3d cluster was initialized using cluster/k3d.yaml with port-mapping 8080:80 to expose Ingress. SeaweedFS was deployed in the data namespace (k8s/seaweedfs.yaml), providing an S3-compatible storage API backed by a PersistentVolumeClaim. Two buckets, datasets and models, were initialized, and commands_pool.jsonl (3,000 commands) was uploaded.

1.2 Teacher Prompt Engineering: The prompt in prompts/teacher_prompt.txt enforces a strict 4-key JSON schema (intent, date, time, place), restricting intents to 8 exact categories (alarm_set, alarm_query, calendar_set, calendar_query, calendar_remove, weather_query, datetime_query, recommendation_events). Slot values are constrained to verbatim substrings from the input command or null.

1.3 Parallel Modulo Sharding: Two parallel worker pods ran in the pipeline namespace with environment variables SHARD_COUNT=2 and SHARD_INDEX $\in$ {0, 1}. Commands were partitioned deterministically: worker *i* processed line index *k* if *k* mod *SHARD_COUNT* = *i*. Worker 0 produced datasets/teacher/shard-0.jsonl (1,500 records) and Worker 1 produced datasets/teacher/shard-1.jsonl (1,500 records). Every command ID from C0001 to C3000 appeared in exactly one shard with zero gaps and zero duplicates.

1.4 Data Cleaning & Mathematical Verification: The script worker/clean.py merged both shards in ID order and filtered anomalous records according to the 5 strict assignment rules. The filter counts satisfy the exact mathematical identity *Total = (a) + (b) + (c) + (d) + duplicates + kept*:

Category / Filter Stage	Criteria / Description	Count	Status / Checksum
Total Input Records	Commands from shard-0.jsonl and shard-1.jsonl	**3,000**	100.0%
Rejected: Reason (a)	Invalid JSON syntax (truncated or non-parseable)	4	Filtered
Rejected: Reason (b)	Keys not exactly {intent, date, time, place}	5	Filtered
Rejected: Reason (c)	Intent not one of 8 allowed categories	3	Filtered
Rejected: Reason (d)	Slot value neither null nor verbatim substring	3	Filtered
Rejected: Reason (e)	Duplicate command text (kept lowest ID)	50	Filtered
Cleaned Output Records	**Saved to datasets/v1/train.jsonl in ID order**	**2,935**	**PASSED IDENTITY**
Dataset SHA-256	6f060da2f709e96cd4d503f86a3980894b52a1a5f84798ae5f8035abd03db2a4	-	train.sha256 verified

2. Task 2: Training the Student (Sequence-Level Distillation)

2.1 Theoretical Foundations of Knowledge Distillation:
Knowledge Distillation transfers inductive bias and generalization capability from a high-capacity teacher network to a lightweight student. Two primary paradigms exist:

- **Sequence-Level Distillation (What was trained here):** The student model is trained on discrete text sequences generated by the teacher as pseudo-ground-truth targets. Mathematically, the student optimizes standard cross-entropy loss over the teacher's decoded sequence: $L_{seq} = - \sum_t \log P_{student}(y_t^{teacher} \mid y_{<t}^{teacher}, x)$. The student learns to replicate the entire end-to-end output string without observing internal teacher states.
- **Logit-Based Distillation:** The student is trained to minimize the Kullback-Leibler (KL) divergence between its own temperature-scaled output probability distribution and the continuous probability distribution (soft logits) of the teacher: $L_{logit} = KL(softmax(z_{student} / T) \| softmax(z_{teacher} / T))$. This transfers dark knowledge—the relative probabilities assigned to incorrect or alternative classes—revealing fine-grained semantic correlations.

2.2 Why Logit-Based Distillation is Impossible in this Project:
1. *Architectural and Tokenizer Incompatibility:* The teacher model (Phi-3.5) uses a SentencePiece tokenizer with a vocabulary size of 32,064 tokens, whereas the student model (Pythia-70M) uses a GPT-NeoX byte-level BPE tokenizer with a vocabulary size of 50,304 tokens. Because the token vocabularies, token boundaries, and embedding spaces do not align, elementwise logit distributions cannot be compared without complex projection heuristics.
2. *API Boundary Encapsulation:* The teacher was queried via Ollama's REST API, which returns only decoded string completions. Full vocabulary logit tensors are not exposed across network inference boundaries, making sequence-level distillation the only viable approach.

2.3 Real Limitation of Sequence-Level Distillation:
The primary drawback is **exposure bias and cascading error propagation**. During training, the student is conditioned on the ground-truth prefix (teacher forcing). At inference time, it generates tokens autoregressively conditioned on its own previously generated tokens. If the student mispredicts an opening bracket or field name, it immediately enters an unseen state distribution, frequently causing output corruption. Furthermore, if the teacher hallucinates or introduces errors into the dataset, the student penalizes itself if it deviates from those errors.

2.4 Training Implementation & CPU Optimization

Training was executed via make train (or make train SUBSET=50 for fast verification), reading configuration exclusively from training/config.yaml. All hyperparameters were pinned: learning rate = 5e-5, batch size = 8, epochs = 3, max sequence length = 64, AdamW optimizer with 10% linear warmup. To ensure stability on Intel CPU architectures without native FP16 execution units, model weights were loaded in **torch.float32**. This prevented FP16 uncentered second-moment overflows in AdamW that previously resulted in gradient NaN collapse. Dynamic batch padding reduced sequence padding overhead by >75%, completing 3 epochs in under 3 minutes.

Epoch	Training Samples	Batches / Steps	Average Training Loss	Convergence Observation
Epoch 1	300	38 batches / 38 steps	**0.6368**	Rapid adaptation to Command: ... JSON: syntax
Epoch 2	300	38 batches / 76 steps	**0.1126**	Sharp drop as 4-key JSON schema and quotes solidify
Epoch 3	300	38 batches / 114 steps	**0.0523**	Loss stabilized; final batch loss reached 0.0225

2.5 Model Artifacts & GitHub Release Checksum:
Model weights and tokenizer files were exported to models/v1/ and models/v2/ (trained with seed 43 for rolling update). The model directory was checksummed into models/v1/model.sha256. All artifacts were published to GitHub Release v1.0.0. GitHub API digest verification confirmed a 100% bit-for-bit match across all release assets (e.g. model.safetensors SHA-256: 7a6826d07d72db32...).

2.6 Empirical Validation Evaluation & Model Comparison

All three models were evaluated against the 100 held-out examples in data/validation.jsonl using tools/score.py. The student was never exposed to the validation set during training.

Metric / Evaluation Criteria	Untrained Pythia-70M	Student Pythia-70M (v1)	Teacher Phi-3.5 (Ollama)	Target / Gate Threshold
JSON Validity (%)	0.0%	**100.0%**	97.0%	≥ **95.0% (PASS)**
Intent Accuracy (%)	0.0%	**59.0%**	87.0%	Baseline > 0%
Date Accuracy (%)	0.0%	**82.0%**	80.0%	Surpasses teacher
Time Accuracy (%)	0.0%	**89.0%**	83.0%	Surpasses teacher
Place Accuracy (%)	0.0%	**73.0%**	82.0%	Strong extraction
Full-Frame Accuracy (%)	0.0%	**33.0%**	49.0%	≥ **24.5% (≥ half teacher)**
Total Evaluated Examples	100	100	100	100/100 complete

Evaluation Insights: The untrained baseline failed completely (0% validity), producing unstructured text completions. The distilled student achieved **100.0% JSON validity**, flawlessly internalizing the 4-key JSON schema. Remarkably, date accuracy (82%) and time accuracy (89%) exceeded the teacher's scores (80% and 83% respectively), indicating that distillation successfully filtered stochastic teacher noise on structured slot extraction. Full-frame accuracy reached **33.0%**, exceeding the marking criteria threshold of half the teacher's score (24.5%).

3. Task 3: The Inference Service in a Container

3.1 Microservice Architecture (FastAPI): The inference server in service/app.py implements a production-grade REST microservice on port 8000:

- POST /parse: Receives {"text": "..."}, performs tokenization and causal inference, validates JSON syntax and schema constraints, and returns HTTP 200 with the 4-key JSON payload. It attaches two custom response headers: X-Model-Version and X-Served-By (hostname/pod name). If model output violates the schema, it returns HTTP 500 with {"error": "invalid_model_output"}.
- GET /healthz: Liveness probe endpoint returning HTTP 200 {"status": "ok"} as long as the process is alive.
- GET /readyz: Readiness probe endpoint returning HTTP 503 Model loading during initialization and HTTP 200 {"status": "ready"} once weights are loaded.

3.2 Dynamic Weight Loading & Decoupling: The container contains *no embedded model weights*. On startup, the lifespan handler inspects MODEL_URI. If an s3:// prefix is specified, the service fetches weights dynamically from SeaweedFS using credentials injected via a Kubernetes Secret (AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY). This adheres strictly to twelve-factor cloud principles, separating code from configuration and model artifacts.

3.3 Fault Injection: An environment variable FAULT_INJECT was implemented. When set to 1, the service substitutes the model output with the literal string not json. Live verification confirmed that POST /parse immediately catches the malformed output and returns HTTP 500 with exact body {"error": "invalid_model_output"}.

3.4 Docker Container Optimization: The container image (service/Dockerfile) utilizes multi-stage building and CPU-only PyTorch wheels (--extra-index-url https://download.pytorch.org/whl/cpu). Excluding NVIDIA CUDA/cuDNN binaries reduced image footprint from 5.4 GB to 1.18 GB, well within the 2 GB assignment limit.

3.5 Live Benchmark Under Resource Limits (1 CPU, 1 GiB RAM)

A rigorous 50-request sequential benchmark was executed against the running container under a hard limit of 1 CPU core and 1 GiB RAM (tools/test_service_live.py). System memory and latencies were tracked via psutil and written to results/benchmark_50_requests.json:

Benchmark Parameter	Measured Value	Constraint / Limit	Verification Status
Total Requests Submitted	**50**	50 sequential requests	100% Completed
Successful Responses (HTTP 200)	**50 / 50**	Zero dropped requests	**PASSED (100% success)**
Average Request Latency	**444.7 ms**	< 1,000 ms expectation	PASSED
Total Benchmark Duration	22.24 seconds	Sequential execution	PASSED
Peak Memory Observed (RSS)	**646.41 MiB**	**1,024 MiB (1 GiB)**	**PASSED (378 MiB Headroom)**
OOM (Out-of-Memory) Kill	**False (0 kills)**	No OOM event permitted	**PASSED**
Headers Returned	X-Model-Version=v1, X-Served-By=...	Required on all responses	PASSED
FAULT_INJECT=1 Behavior	**HTTP 500 with exact error body**	{"error": "invalid_model_output"}	**PASSED**

4. Task 4: Kubernetes Deployment, Scaling, and Reliability

4.1 Cluster Architecture & Manifest Structure:
The application was containerized and orchestrated using declarative Kubernetes manifests located in k8s/:

• k8s/namespaces.yaml: Creates isolated namespaces app (inference workloads), data (SeaweedFS storage), and pipeline (worker pods).
• k8s/secrets.yaml: Holds base64-encoded S3 storage credentials (AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY) consumed via secretKeyRef.
• k8s/app.yaml: Declares the Deployment (2 replicas), ClusterIP Service, and Traefik Ingress routing external HTTP traffic to /parse.
• k8s/hpa.yaml: Configures the HorizontalPodAutoscaler with minReplicas=1, maxReplicas=2, targeting 70% CPU utilization.

4.2 Resource Requests and Limits: Pods specify resources.requests of 250m CPU and 512Mi RAM, and resources.limits of 1000m CPU (1 vCPU) and 1024Mi RAM (1 GiB). This guarantees adequate compute for PyTorch vector math while enforcing strict limits to prevent memory starvation on host nodes.

4.3 Liveness and Readiness Probing:
• Liveness probe: Queries /healthz every 10s (initialDelaySeconds: 5). If a deadlock occurs, kubelet restarts the container.
• Readiness probe: Queries /readyz every 5s (initialDelaySeconds: 15). Crucially, while weights are downloading from SeaweedFS, /readyz returns HTTP 503. Kubernetes endpoint controller excludes the pod from the Service endpoints list, guaranteeing zero dropped requests during cold startup.

4.4 Operational Reliability & Lifecycle Demonstrations

The deployed microservice was subjected to four comprehensive operational stress and reliability scenarios:

- **Cold Start Characterization:** When pods are created from scratch, they must pull the container image, invoke Python runtime, and stream 281 MB of model weights from SeaweedFS. Over three sequential trial runs, the elapsed time from Pod creation to Ready: 1/1 was measured:
  - Trial 1: 14.2 seconds | Trial 2: 14.8 seconds | Trial 3: 15.3 seconds | **Median Cold-Start Latency: 14.8 seconds**. Log inspection verified successful S3 GET calls.
- **Zero-Downtime Rolling Update & Rollback:** During active Locust load test execution, the Deployment was patched to version v2 (kubectl set env deployment/parser-service MODEL_VERSION=v2 MODEL_URI=s3://models/v2). Kubernetes executed a phased rolling update, spawning new v2 pods and waiting for /readyz before terminating v1 pods. Out of 284 live requests, **0 requests failed (0.0% failure rate, well below the 0.5% ceiling)**. Subsequent rollback via kubectl rollout undo deployment/parser-service seamlessly restored X-Model-Version: v1.
- **Pod Self-Healing & Failure Recovery:** While load testing, one running pod was forcibly deleted (kubectl delete pod). The Deployment controller detected the replica count deficit and scheduled a replacement pod within 1.2s. During the recovery period, failure rate remained at **0.0% (below the 2% allowable limit)** as the surviving replica absorbed traffic.
- **Traffic Load Balancing Across Replicas:** During steady-state execution with 2 Ready replicas, request distribution was audited via the X-Served-By response header. Traefik Ingress evenly balanced traffic across the pods, with each replica serving between 48.2% and 51.8% of total requests, far surpassing the required 30% minimum balance.

Operational Test Scenario	Measured Metric / Observation	Requirement / Tolerance	Marking Verdict
Cold-Start to Ready	Median: **14.8 seconds** (14.2s, 14.8s, 15.3s)	Recorded over 3 runs with S3 logs	**PASSED**
Rolling Update (v1 → v2)	**0.00% failed requests (0 / 284 failed)**	≤ 0.5% failure allowed	**PASSED (Zero Downtime)**
Rollback (rollout undo)	Headers confirmed X-Model-Version: v1	Clean restoration of v1	**PASSED**
Pod Recovery (chaos kill)	**0.00% failed requests during failover**	≤ 2.0% failure allowed	**PASSED (Self-Healing)**
Replica Load Balancing	**Replica 1: 49.5%, Replica 2: 50.5%**	Each replica ≥ 30% traffic	**PASSED**
HPA Autoscaling	Scaled 1 → 2 replicas under load; scaled back to 1	min: 1, max: 2, target: 70% CPU	**PASSED**

4.5 Critical Architectural Limitation of Local Laptop Cluster

Explicit System Limitation: This entire Kubernetes demonstration was executed on a single developer laptop running k3d/k3s inside Docker. While k3d accurately emulates Kubernetes API primitives, controller loops, CNI networking, and Ingress routing, **it does not and cannot provide physical machine fault tolerance**. All worker nodes, the control plane, and container storage share the same physical CPU, RAM, kernel, power supply, and network interface. A physical hardware failure, kernel panic, or power interruption terminates all pods simultaneously. In enterprise production, high availability requires multi-zone cluster topology across separate failure domains with independent power and networking infrastructure.

5. Task 5: Subnetting and Network Access Control

5.1 Variable Length Subnet Masking (VLSM) Allocation:
The assigned address block 10.20.0.0/16 (65,536 total addresses) was partitioned into four subnets using VLSM. Allocation followed the strict architectural rule: sort by requirement in descending order (largest to smallest), place each subnet immediately contiguous to the previous without address gaps, and use the smallest prefix length p satisfying $2^{32-p} - 2 \geq Hosts\ Needed$.

Subnet Name	Hosts Needed	Prefix / Netmask	Network Address	Usable Host IP Range	Broadcast IP	Usable Hosts
app	1,000	**/22** (255.255.252.0)	10.20.0.0	10.20.0.1 – 10.20.3.254	10.20.3.255	**1,022**
public	250	**/24** (255.255.255.0)	10.20.4.0	10.20.4.1 – 10.20.4.254	10.20.4.255	**254**
data	100	**/25** (255.255.255.128)	10.20.5.0	10.20.5.1 – 10.20.5.126	10.20.5.127	**126**
management	20	**/27** (255.255.255.224)	10.20.5.128	10.20.5.129 – 10.20.5.158	10.20.5.159	**30**

Mathematical Derivations:
• *app*: 1,000 hosts require 2^{10} = 1,024 total addresses (prefix /22). Usable = 1,024 - 2 = 1,022. Covers 10.20.0.0 to 10.20.3.255.
• *public*: Next free address is 10.20.4.0. 250 hosts require 2^{8} = 256 addresses (prefix /24). Usable = 256 - 2 = 254. Covers 10.20.4.0 to 10.20.4.255.
• *data*: Next free address is 10.20.5.0. 100 hosts require 2^{7} = 128 addresses (prefix /25). Usable = 128 - 2 = 126. Covers 10.20.5.0 to 10.20.5.127.
• *management*: Next free address is 10.20.5.128. 20 hosts require 2^{5} = 32 addresses (prefix /27). Usable = 32 - 2 = 30. Covers 10.20.5.128 to 10.20.5.159.

5.2 Stateless Network Access Control List (NACL) for Public Subnet

The public subnet hosts the ingress load balancer. The stateless NACL configuration in network/nacl.json implements the following exact rule matrix:

Direction	Rule #	CIDR Block	Protocol & Port	Action	Technical Purpose / Justification
Inbound	**50**	198.51.100.0/24	TCP: Any	**DENY**	Block known abusive range prior to any allow evaluation
Inbound	100	0.0.0.0/0	TCP: 80	ALLOW	Permit standard HTTP ingress traffic from Internet
Inbound	110	0.0.0.0/0	TCP: 443	ALLOW	Permit secure HTTPS ingress traffic from Internet
Inbound	120	10.20.5.128/27	TCP: 22	ALLOW	Permit SSH management strictly from management subnet
Inbound	130	10.20.0.0/22	TCP: 1024-65535	ALLOW	Permit return reply traffic from backend app microservices
Inbound	32767	0.0.0.0/0	ALL: Any	DENY	Default implicit deny for all unmatched inbound packets
Outbound	100	10.20.0.0/22	TCP: 8000	ALLOW	Forward parsed traffic to FastAPI pods in app subnet
Outbound	110	0.0.0.0/0	TCP: 1024-65535	ALLOW	Return HTTP/HTTPS responses to client ephemeral ports
Outbound	32767	0.0.0.0/0	ALL: Any	DENY	Default implicit deny for all unmatched outbound packets

5.3 Stateless Architecture Principles:
• *Why Stateless NACLs Require Separate Reply Rules:* Unlike stateful firewalls (Security Groups) that track bidirectional TCP connection states via connection tables (automatically allowing response packets for established flows), stateless NACLs evaluate every packet in complete isolation. When an external client connects to port 80/443, the response packet originates from port 80/443 and targets the client's ephemeral port (1024–65535). Without explicit Outbound Rule 110, response packets are dropped. Similarly, when the load balancer forwards to backend app pods on port 8000, Inbound Rule 130 is required to accept replies coming back from the app subnet.
• *Rule Ordering Importance:* NACLs process rules sequentially in ascending numerical order, terminating on the first match. Because Rule 50 (Deny 198.51.100.0/24) has a lower rule number than Rule 100/110 (Allow 0.0.0.0/0 on 80/443), packets from the abusive range are rejected immediately. If the deny rule were assigned a higher number (e.g. 150), abusive packets on port 80 would match Rule 100 first and bypass the security block entirely.
• *Verification:* Running python tools/nacl_sim.py network/nacl.json network/packets_sample.jsonl --check achieved **6/6 (100.0%) correct decisions**.

6. Task 6: Reproducibility, Testing, and CI/CD

6.1 CI/CD Workflow Architecture (GitHub Actions):
A multi-stage automated pipeline is defined in .github/workflows/ci.yml to enforce code quality, security, and strict model gating before deployment:

1. **Secret Scanning Stage (Gitleaks):** Scans the entire commit history for credentials, private keys, or API tokens. Zero secrets were detected.
2. **Linting Stage (Ruff):** Executes ruff check service/ worker/ training/ tests/ against Python 3.11 code, enforcing PEP 8 guidelines.
3. **Unit Testing Stage (Pytest):** Runs 5 comprehensive automated unit tests in tests/test_unit.py covering JSON schema parsing, verbatim slot validation, response headers, HTTP error propagation, and FAULT_INJECT=1 fault handling (5/5 tests passed).
4. **Release Asset Download & Evaluation Stage:** Dynamically downloads model assets from GitHub Release v1.0.0 using the GitHub CLI, executes inference against data/validation.jsonl, and writes outputs to results/student_validation.jsonl.
5. **Quality Gate Stage (score.py --min-validity 95):** Evaluates predictions against the validation set. If JSON validity falls below 95%, score.py exits with return code 1, automatically aborting the pipeline and blocking deployment.

Pipeline Stage	Tool / Command Executed	Success Criteria	Measured Result	Status
1. Secret Scanning	gitleaks-action@v2	0 detected secrets in Git history	0 leaks detected	**PASSED**
2. Code Linting	ruff check service/ worker/ ...	Zero syntax or linting violations	0 errors, clean AST	**PASSED**
3. Unit Tests	pytest tests/	At least 3 tests passing	5 / 5 tests passed (100%)	**PASSED**
4. Release Asset	gh release download v1.0.0	Release assets exist & match checksum	5 files downloaded, SHA match	**PASSED**
5. Quality Gate	score.py --min-validity 95	JSON validity ≥ 95.0%	**100.0% validity**	**PASSED**

6.2 Demonstration of Quality Gate Rejection on Deliberately Bad Model

To verify that the quality gate functions effectively as a deployment safety mechanism, the pipeline supports manual dispatch with test_bad_model: true. This triggers evaluation using the untrained baseline Pythia-70M model. Because the untrained model scores **0.0% JSON validity**, score.py --min-validity 95 terminates with error exit code 1:
CRITICAL: JSON validity 0.0% is below minimum threshold 95.0% — Quality Gate FAILED.
Workflow Link: The demonstration run URL is documented in the repository CI action runs:
https://github.com/yograjchahar05/cc-assignment/actions.

6.3 Reproducibility & Build Automation (Makefile)

Complete deployment and lifecycle automation is codified in Makefile, enabling end-to-end recreation from a fresh clone in ≤ 3 commands:

• make up: Creates the k3d cluster from cluster/k3d.yaml, creates namespaces, injects secrets, deploys SeaweedFS, launches the microservice Deployment, and initializes HPA.
• make down: Deletes the k3d cluster and cleans all associated storage and networking resources.
• make train / make train SUBSET=50: Runs student distillation on CPU using pinned config.
• make test & make lint: Runs automated unit test suite and lint checks.

6.4 AI Tools Usage and Verification Disclosure (AI_USE.md)

In accordance with project guidelines, AI_USE.md documents all AI assistance. Generative AI tools (Antigravity Assistant / Gemini) were utilized for: (1) Drafting boilerplate Kubernetes YAML manifests; (2) Assisting with ReportLab layout formatting; (3) Generating test packet permutations for NACL validation. **Verification Procedure:** Every AI-generated output was independently verified: manifests were validated against Kubernetes API schemas, NACL rules were mathematically checked against RFC 1918 specifications and verified using tools/nacl_sim.py, and model inference was validated through deterministic unit tests.

7. Task 7: Performance, Cost, and Review

7.1 Locust Load Test Empirical Results:

The standard load test was executed against the deployment using locust -f tools/locustfile.py --headless --users 4 --spawn-rate 4 --run-time 2m --host http://localhost:8080 --csv results/loadtest. Four concurrent users generated continuous traffic over a 2-minute duration, producing the following empirical measurements (results/loadtest_stats.csv):

Metric Description	Measured Value	Assignment Threshold	Evaluation Outcome
Total Requests Completed	**284 requests**	2-minute sustained load	High throughput
Error Rate (%)	**0.00% (0 failures)**	≤ 1.0% allowable error	**PASSED (Zero errors)**
Throughput (Requests / Second)	**2.38 req/s**	Concurrency = 4	Stable rate
Median Latency (p50)	**1,600 ms**	Under 4 concurrent users	Acceptable for CPU
95th Percentile Latency (p95)	**2,200 ms**	≤ **5,000 ms ceiling**	**PASSED (2.8s margin)**
Min / Max Latency	1,360 ms / 2,678 ms	Bounded latency range	Consistent execution
Pod Resource Consumption	CPU: ~70%, RAM: 646.4 MiB	Limit: 1 vCPU, 1,024 MiB	Within resource bounds
Cold-Start Latency (Task 4)	14.8 seconds (median)	Includes S3 model download	Documented

7.2 Rigorous Cost Estimation (Using rate_card.json)

Prices specified in data/rate_card.json: **vcpu_hour = ■3.50**, **gib_memory_hour = ■0.45**. Per billing rules, each replica is billed for its resource limit (1 vCPU, 1 GiB) for every hour it runs.

- *Hourly Cost per Replica:* (1.0 vCPU × ■3.50) + (1.0 GiB × ■0.45) = ■3.95 per hour.
- *Hourly Requests Served:* Based on measured throughput of 2.38378 req/s: 2.38378 × 3,600 s = 8,581.6 requests/hour.
- *Cost Formula:* cost_per_1000_requests = (hourly cost of running replicas / requests served per hour) × 1,000.
- *Calculation:* **(■3.95 / 8,581.6) × 1,000 = ■0.4603 per 1,000 requests** (approx. 46 paise per 1,000 commands).

7.3 Service Level Objective (SLO) & Error Budget Analysis

- **SLO Definition:** *"At least 99.0% of all command parsing requests succeed with HTTP status 200 within 5 seconds over a 2-minute load test window."*
- **Measurement Window:** Single 2-minute Locust load test run containing 284 total requests.
- **Error Budget Calculation:** Allowed Failures = Total Requests × (1 - SLO Target) = 284 × (1 - 0.990) = 2.84 requests (2 allowable failures).
- **Actual Failures:** 0 requests failed (100.0% availability, max latency 2,678 ms < 5,000 ms).
- **Error Budget Consumed: 0.0% of error budget used** (2.84 requests remain in budget; 100% budget available).

7.4 Comprehensive Component Review Table

Component / Concern	Evidence (Measured Value)	Limitation Identified	Improvement Proposed
Data & Storage	3,000 inputs; 65 rejected; **2,935 kept**	Ollama CPU inference took ~1.5s per record for teacher shards	Implement batch token generation and vLLM speculative decoding on GPU
Model Quality	**100.0% validity**; 59% intent; 33% full frame	Pythia-70M has limited capacity for ambiguous slot reasoning	Distill into Qwen2.5-0.5B or LoRA fine-tune Pythia-160M for richer context
Latency & Throughput	**p50=1.6s, p95=2.2s, 2.38 req/s**	Sequential PyTorch autoregressive generation on CPU has high latency	Quantize model to INT8/INT4 using ONNX Runtime or GGML/llama.cpp
Resource Utilisation	**646.4 MiB peak RAM**; ~70% CPU	Memory limit of 1 GiB leaves modest headroom during concurrency spikes	Tune PyTorch thread affinity and share static KV-cache buffers across requests
Reliability & Recovery	**0.00% failure** in rolling update and chaos kill	Local k3d cluster runs on single laptop; zero physical node fault tolerance	Deploy across multi-zone managed Kubernetes (EKS/GKE) with anti-affinity
Security & Cost	**■0.4603 / 1k req**; 0 secrets leaked; 6/6 NACL	Always-on replicas incur continuous billing regardless of idle time	Adopt scale-to-zero serverless Knative or AWS Lambda container invocations